SELF-DIRECTED LEARNING PLAN

AI Engineering Roadmap

A seven-phase, project-driven path from API foundations to multi-agent orchestration and capstone integration — built on direct API calls and mechanical understanding rather than framework abstractions.

7	~28	~190	5
PHASES	WEEKS	HOURS	PROJECTS

> *"Skip premature abstractions. Learn the mechanics by building with direct API calls — frameworks come later, once the patterns underneath them are already understood."*

Roadmap at a Glance

Updated — added Phase 4.5 (multi-agent orchestration) and Phase 6 (frameworks)

P1	P2	P3	P4	P4.5	P5	P6
Wks 1–3	Wks 4–7	Wks 8–13	Wks 14–17	Wks 18–20	Wks 21–25	Wks 26–28

WEEKS 1–3 · ~18 HOURS

1 Smart Expense Tracker DONE

Closing API foundation gaps — auth, request/response handling, error states, persistence.

CORE LEARNING TARGETS

- Structuring a FastAPI app with proper request/response models
- Serving with Gunicorn — process management basics
- Designing clean error states instead of raw stack traces
- First contact with the Claude API for unstructured-to-structured parsing

Project: An expense tracker that takes free-text entries ("$12 lunch with Sam") and parses them into structured records via the Claude API, served through FastAPI + Gunicorn.

RESOURCES & LEARNING MATERIAL

↗ **Anthropic Academy — Building with the Claude API** OFFICIAL
Skilljar, ~8 hrs. The deepest official course on request structure, error handling, and API patterns.

↗ **FastAPI official docs — Tutorial** OFFICIAL
fastapi.tiangolo.com/tutorial — work through request bodies, response models, and error handling sections specifically.

↗ **Gunicorn docs — Design & Deployment** OFFICIAL
docs.gunicorn.org — worker types and process management; short, no need for more than this.

WEEKS 4–7 · ~22 HOURS

2 Natural Language Calendar Agent DONE

Agentic tool loops, built with direct Anthropic API calls — no LangChain.

CORE LEARNING TARGETS

- Mechanical understanding of the tool-use loop: `tool_use` → execute → `tool_result` → continue
- Writing tool descriptions as prompts, not documentation
- Chaining as an emergent model decision, not explicit orchestration code
- Correct placement of human-in-the-loop confirmation (system prompt *and* tool description)
- Safe conversation-history trimming (avoiding the 400 Bad Request from orphaned tool_result blocks)

Project: A Google Calendar agent (get_events, find_free_slots, create/update/cancel_event, get_weather) with a CLI front-end and a Telegram bot stretch goal — confirmed working, including a real encounter with the history-trimming 400 error.

RESOURCES & LEARNING MATERIAL

↗ **Anthropic docs — Tool use overview** OFFICIAL
docs.claude.com/en/docs/build-with-claude/tool-use — the canonical reference for tool_use/tool_result block structure.

↗ **Claude Cookbook — Tool use & function calling** OFFICIAL
github.com/anthropics/claude-cookbooks — runnable notebooks showing the full loop, including multi-tool chaining.

↗ **Google Calendar API — Python quickstart** OFFICIAL
developers.google.com/calendar/api/quickstart/python — covers OAuth consent screen, scopes, and token refresh.

WEEKS 8–13 · ~32 HOURS

3 AI Support Agent: RAG + Eval Harness IN PROGRESS

Vector databases introduced contextually, exactly when the project needs them.

CORE LEARNING TARGETS

- Chunking strategy and embedding generation — the mechanics, not just calling an API
- ChromaDB locally → similarity search → relevance thresholds
- Building an eval harness: retrieval precision/recall, groundedness checks, hallucination detection
- Where RAG grounding constraints belong (same "close to the decision point" principle from tool descriptions)

Project: A support agent answering grounded questions from a real document corpus, with a working eval harness measuring whether retrieved chunks actually support the generated answer. Optional cloud stretch: Supabase pgvector.

RESOURCES & LEARNING MATERIAL

↗ **Claude Cookbook — RAG guide** OFFICIAL
github.com/anthropics/claude-cookbooks/.../retrieval_augmented_generation/guide.ipynb — naive RAG pipeline plus a dedicated section on evaluating retrieval vs. end-to-end performance separately.

↗ **Claude Cookbook — Contextual retrieval guide** OFFICIAL
platform.claude.com/cookbook/capabilities-contextual-embeddings-guide — improving chunk relevance with contextual embeddings; read after the basic RAG guide, not before.

↗ **ChromaDB docs — Getting Started** OFFICIAL
docs.trychroma.com — local persistence, collections, and similarity search basics; you need only the first few pages.

↗ **Anthropic Academy — AI Fluency / Evaluations track** OFFICIAL
Skilljar — if a course on building eval suites is listed, it's the most relevant addition once retrieval is working.

WEEKS 14–17 · ~24 HOURS

4 MLOps Sprint

Deploy and monitor the Phase 3 RAG agent — taking it from script to service.

CORE LEARNING TARGETS

- Containerizing the RAG agent for deployment
- Basic monitoring: latency, error rates, eval-score drift over time
- Logging tool calls and retrieval results for post-hoc debugging
- What "production-ready" actually requires beyond a working notebook

---

Project: The Phase 3 RAG agent deployed as a monitored service, with dashboards or logs showing real usage and eval metrics over time.

RESOURCES & LEARNING MATERIAL

↗ **Docker docs — Get Started** OFFICIAL
docs.docker.com/get-started — just enough to containerize a FastAPI service correctly.

↗ **Anthropic docs — Monitoring & usage** OFFICIAL
docs.claude.com — check the platform usage/cost tracking pages for what's available out of the box before building custom logging.

↗ **Prometheus + Grafana — Get Started guide** COMMUNITY
The standard lightweight stack for self-hosted latency/error dashboards; overkill is easy here, keep it to 2–3 panels.

WEEKS 18–20 · ~16 HOURS

4.5 Multi-Agent Orchestration NEW

Sub-agent delegation and context management — via direct API calls, before any framework.

CORE LEARNING TARGETS

- **Orchestrator/worker pattern:** a "delegate" tool whose implementation is itself a full nested agent loop
- **Context isolation vs. sharing:** does a sub-agent get full parent history, or a scoped summary?
- **Result surfacing:** collapsing a sub-agent's internal multi-turn loop into a single tool_result block
- **Failure propagation:** what the orchestrator sees when a sub-agent errors several levels deep

---

Project: An orchestrator agent that nests the Phase 2 calendar agent and Phase 3 RAG agent as sub-agents — handling requests like "check my calendar for conflicts this week and summarize support tickets about scheduling issues," which forces real delegation and result-merging.

RESOURCES & LEARNING MATERIAL

↗ **Claude Cookbook — Sub-agents notebook** OFFICIAL
github.com/anthropics/claude-cookbooks — using a smaller model as a sub-agent under a larger orchestrator; the closest official example to this phase's project.

↗ **Claude Code docs — Subagents** OFFICIAL
docs.claude.com/en/docs/claude-code/sub-agents — even though this describes Claude Code's own subagents, the context-isolation model it documents is conceptually identical to what you're hand-building.

↗ **Anthropic Academy — Subagents course** OFFICIAL
Skilljar, ~30–60 min — short, but useful for vocabulary before you build the pattern yourself from scratch.

WEEKS 21–25 · ~30 HOURS

5 MCP Capstone: Life Admin Hub

Connecting all prior projects through the Model Context Protocol.

CORE LEARNING TARGETS

- Exposing the calendar agent, RAG agent, and expense tracker as MCP servers
- Composing them into one coherent assistant via MCP rather than ad-hoc glue code
- Applying Phase 4.5's orchestration patterns to a real multi-project system
- End-to-end integration testing across services with different deployment states

---

Project: "Life Admin Hub" — a single assistant that, via MCP, can manage your calendar, answer support-style questions through RAG, and log expenses, unifying every prior phase into one system.

RESOURCES & LEARNING MATERIAL

↗ **Model Context Protocol — Official docs** OFFICIAL
modelcontextprotocol.io — start with "Core Architecture" then "Build an MCP Server," in that order.

↗ **Anthropic docs — MCP quickstart** OFFICIAL
docs.claude.com/en/docs/claude-code/mcp — connects an MCP server end to end; the fastest path to a working first server.

↗ **Claude Cookbook — MCP examples** OFFICIAL
github.com/anthropics/claude-cookbooks — check the mcp/ directory for server examples close to your use case.

WEEKS 26–28 · ~14 HOURS

6 Frameworks: LangGraph & n8n NEW

Now that the mechanics are second nature, learn the abstractions that wrap them.

CORE LEARNING TARGETS

- Mapping LangGraph's graph/state/node abstractions onto the tool loop you already built by hand
- Recognizing what LangGraph buys you (state persistence, visualization, checkpointing) versus what it hides
- n8n as a visual automation layer — connecting your existing agents into workflows without glue code
- Judging, case by case, when a framework is worth the abstraction cost in real projects

Project: Re-implement one existing agent (e.g. the orchestrator from Phase 4.5) in LangGraph, then build an n8n workflow that triggers it — comparing both against the hand-built version for clarity, debuggability, and development speed.

RESOURCES & LEARNING MATERIAL

↗ **LangGraph official docs — Tutorials** OFFICIAL
langchain-ai.github.io/langgraph — start with the "Quickstart" then "Common workflows"; you'll recognize the tool loop immediately.

↗ **LangGraph — Multi-agent concepts page** OFFICIAL
Read this specifically against your Phase 4.5 build — it's the best way to see what the framework abstracts vs. what you already did by hand.

↗ **n8n official docs — Workflow basics** OFFICIAL
docs.n8n.io — nodes, triggers, and the HTTP Request node are the only things you need to connect your existing agents.

↗ **n8n — AI Agent node documentation** OFFICIAL
Covers wiring n8n directly to the Claude API and to MCP servers from Phase 5.

Guiding Principles

What's carried through every phase of this roadmap

Mechanics before abstraction	Every pattern (tool loops, RAG, orchestration) is built once with direct API calls before any framework is introduced. Frameworks are learned last, in Phase 6, once there's a hand-built mental model to compare them against.
Contextual introduction	Tools like vector databases are introduced exactly when a project needs them — not front-loaded. This is why ChromaDB lives in Phase 3, not Phase 1.
Confirmation at the decision point	

	Constraints (human-in-the-loop confirmation, RAG grounding rules) belong as close to the model's decision point as possible — in tool descriptions, not just system prompts.
Credentials vs. self-direction	Paid courses are skipped in favor of free, high-quality resources (Anthropic's Skilljar curriculum) unless a specific credential has clear employer value.

Skip / Defer / Learn Decisions

LangChain / LangGraph	**Deferred to Phase 6.** Abstracts away the tool-loop mechanics that Phases 2–4.5 are designed to teach directly.
n8n	**Deferred to Phase 6.** A productivity multiplier for someone who already has working agents to connect — not a foundational tool.
Vector databases	**Learn in Phase 3.** Introduced contextually when the RAG project creates a real need for similarity search.
Multi-agent orchestration	**Learn in Phase 4.5.** Mechanical, hand-built — sub-agent delegation and context management before any orchestration framework.
BITS Pilani GenAI course	**Skipped.** Assessed as not worth enrolling in given demonstrated self-direction; free resources cover the same ground.